



SYSTEM DESIGN FUNDAMENTALS

CORE PRINCIPLES

Scalability

Vertical Scaling (Scale Up): Add more power to existing machines (CPU, RAM). Simple but has limits.

Horizontal Scaling (Scale Out): Add more machines. Unlimited growth but more complex.

Reliability & Availability

Reliability: System performs consistently without failure.

Availability: Percentage of operational time (99.99% = 52.6 min downtime/year).

Fault tolerance: Redundancy, graceful degradation, automatic failover.

Performance

Latency: Time between request and response. Critical for user experience.

Throughput: Volume of operations processed. Important for concurrent users.

Consistency (CAP Theorem)

Distributed systems can only guarantee 2 of 3:

- **Consistency:** All nodes see same data
- **Availability:** Every request gets response
- **Partition Tolerance:** Works despite network failures

KEY BUILDING BLOCKS

Load Balancer

Distributes requests across servers. Prevents overload, improves availability, enables scaling. Algorithms: round-robin, least connections, weighted distribution.

Caching

Stores frequently accessed data in fast storage. **Cache hit:** immediate return. **Cache miss:** query database, store copy. Tools: Redis, Memcached, CDNs.

Databases

SQL: Structured tables, ACID properties, complex queries. Best for structured data (PostgreSQL, MySQL).

NoSQL: Flexible schemas, prioritizes scalability. Types: document (MongoDB), key-value (Redis), column-family (Cassandra).

Message Queue

Asynchronous producer-consumer communication. Provides reliability, scalability, load smoothing, loose coupling. Examples: RabbitMQ, Kafka, Amazon SQS.

PATTERNS & TRADE-OFFS

Monolithic vs. Microservices

Monolithic: Single unit. Simple to develop/deploy, but hard to scale specific features.

Microservices: Independent services. Offers independent scaling and faster deployment, but increases operational complexity.

Understanding Trade-offs

System design balances competing goals:

- **Consistency vs. Availability** (CAP Theorem)
- **Performance vs. Cost:** More resources = better performance
- **Simplicity vs. Flexibility:** Easy to use vs. configurable

The Design Process

1. Understand Requirements: Gather functional/non-functional requirements, define scale and metrics.

2. High-Level Design: Create architectural overview, identify building blocks, draw data flow.

3. Detailed Design: Define data models, APIs, address bottlenecks, plan for failures.



GOAL: Build scalable, reliable, and performant systems that meet business needs. Start simple, measure needs, and evolve as requirements grow.



HORIZONTAL VS VERTICAL SCALING

HORIZONTAL SCALING (Scaling Out)

Add more machines to the pool. Distribute workload across multiple servers.

Pros:

- ✓ **Elasticity:** Easy to add or remove machines as needed
- ✓ **Fault Tolerance:** No single point of failure - if one server fails, others continue
- ✓ **Cost-Effective at Scale:** Use commodity hardware for massive scale

Cons:

- ✗ **Complexity:** Requires load balancing and distributed system management
- ✗ **Network Overhead:** Latency and communication costs between nodes

Best for:

Massive scale, high availability, distributed applications (e.g., Web/App Servers, NoSQL Databases)

VERTICAL SCALING (Scaling Up)

Add more power (CPU, RAM, storage) to existing machine. Make the server bigger and more powerful.

Pros:

- ✓ **Simplicity:** Easier to manage - no load balancer needed
- ✓ **No Network Overhead:** All communication happens within one machine
- ✓ **Lower Initial Cost:** More economical for small scale operations

Cons:

- ✗ **Limited Scalability:** Hardware has physical limits - can't scale infinitely
- ✗ **Single Point of Failure:** If the server goes down, everything stops
- ✗ **Downtime for Upgrades:** Requires system restart and potential service interruption

Best for:

Simplicity, initial growth, monolithic applications (e.g., Traditional RDBMS, small-to-medium workloads)



KEY TAKEAWAY

Horizontal Scaling:

More machines → Elastic, complex, distributed. Ideal for cloud-native applications requiring high availability.

Vertical Scaling:

Bigger machine → Simple, limited, centralized. Best for traditional applications with predictable growth.

Decision Factors:

Choose based on scalability requirements, budget constraints, application architecture, and team expertise.



FRONTEND VS BACKEND

Modern applications are divided into two main parts: **Frontend** (what users see and interact with) and **Backend** (the logic and data that power the application). They work together through APIs to create seamless user experiences.

FRONTEND (Client-Side)

The presentation layer that runs in the user's browser or mobile app. Focuses on visual design, user interaction, and client-side logic.

Core Technologies

HTML: Structure

CSS: Styling

JavaScript: Behavior

React/Vue/Angular:
Frameworks

What It Does

Frontend handles everything users directly interact with. When you click a button, fill a form, or see an animation, that's frontend in action. It sends requests to the backend and displays the results in a user-friendly way.

Key Responsibilities

- **UI/UX Design:** Visual layout and user experience
- **Responsive Design:** Adapt to different screen sizes
- **Performance:** Fast loading and smooth interactions
- **State Management:** Track user actions and data in-app

Frontend Developer • UI/UX Designer

BACKEND (Server-Side)

The application layer that runs on servers. Handles business logic, data processing, authentication, and database operations that users don't see.

Core Technologies

Node.js/Python/Java:
Languages

PostgreSQL/MongoDB:
Databases

REST/GraphQL: APIs

Apache/Nginx: Servers

What It Does

Backend processes requests from the frontend, applies business rules, interacts with databases, and returns data. It ensures data integrity, enforces security, and handles complex calculations that shouldn't happen on the client side.

Key Responsibilities

- **Business Logic:** Core application rules and workflows
- **Security:** Authentication, authorization, encryption
- **Scalability:** Handle growing user load efficiently
- **Data Management:** Store, retrieve, and process data

Backend Developer • DevOps Engineer

How They Work Together

Aspect:	Frontend	Backend
Focus	User Experience	Logic & Data
Execution	Client's Device (Browser/App)	Server/Cloud
Data Handling	Display & Input Collection	Storage & Processing
Security	Limited (Client-Side Validation)	Critical (Server-Side Verification)



Communication: Frontend sends API requests (HTTP/HTTPS) → Backend processes & queries database → Backend returns JSON/data → Frontend displays results to user



CLIENT-SERVER ARCHITECTURE

A distributed computing model where **clients** (user devices) request services and resources from centralized **servers** that process requests and manage data. This is the foundation of most modern web and mobile applications.

CLIENT SIDE

Clients are user-facing devices (laptops, phones, tablets) that initiate requests. They run the user interface and send requests to servers when users interact with the application.

Client Responsibilities

Request Initiation: User actions trigger HTTP/HTTPS requests to the server

UI Rendering: Display data received from server in user-friendly format

Input Handling: Collect and validate user input before sending to server

SERVER SIDE

Servers are powerful computers running continuously, waiting for client requests. They contain multiple components working together to process requests and manage application resources.

Server Components

Web Server: Handles HTTP requests (Apache, Nginx)

App Server: Executes business logic and application code

Database: Stores and retrieves persistent data (SQL/NoSQL)



Request-Response Cycle

1

Client Request

User action triggers HTTP request to server

2

Server Processing

Web server routes to app server for logic execution

3

Data Access

App server queries database if data needed

4

Server Response

Server sends data back (HTML, JSON, XML)



Key Characteristics

Centralized Resources

All data and processing power concentrated on servers, making management and security easier

Client Initiates Requests

Communication always starts from client side; servers respond to requests only

Server Manages Data

Single source of truth for data; clients display data but don't own it

Scalable Server-Side

Easy to add more servers to handle increased load without changing clients

✓ Advantages of Client-Server Architecture

Centralized Control

Data and security managed in one place

Easy Maintenance

Update server without touching clients

Resource Sharing

Multiple clients access same resources

Scalability

Add more servers as demand grows

Security

Sensitive data stays on protected servers

Cross-Platform

Different clients (web, mobile) use same server



DATABASES - ADVANTAGES & CHALLENGES

Databases are organized systems for storing, managing, and retrieving data efficiently. They power everything from small apps to enterprise systems, offering powerful capabilities alongside important considerations.

✓ ADVANTAGES

Structured Data

Data organized in tables or documents with relationships. Schema enforcement ensures consistency and makes querying intuitive.

Data Integrity & Security

Built-in constraints ensure accuracy. Access controls and encryption protect data from unauthorized access and loss.

Efficient Retrieval

Query languages and indexing enable quick access to specific information. Complex queries filter and aggregate data in milliseconds.

Scalability

Handles growing data volumes through vertical or horizontal scaling. Can manage terabytes to petabytes of data efficiently.

Concurrent Access

Multiple users can read and write simultaneously. ACID properties ensure data consistency with concurrent operations.

✗ CHALLENGES

Complexity

Designing optimal schemas requires expertise. Setup and maintenance demand specialized knowledge and performance tuning skills.

Cost

Requires expensive hardware, software licenses, and skilled administrators. Cloud databases add ongoing operational costs.

Single Point of Failure

Database failure stops the entire application. Requires redundancy, replication, and backup strategies to minimize downtime.

Performance Issues

Can become a bottleneck under high load. Requires monitoring, query optimization, indexing, and sometimes caching layers.

Data Migration

Moving data between databases is complex and risky. Downtime and data integrity concerns make migrations challenging.



KEY TAKEAWAY

Balance Trade-offs

Weigh benefits against complexity and cost based on your specific needs

Choose Right Type

SQL for structured data with relationships; NoSQL for flexibility and scale

Plan for Growth

Design with scalability, backups, and maintenance strategies from the start



HOW A LOAD BALANCER WORKS

A **load balancer** is a critical component that sits between users and backend servers, distributing incoming traffic across multiple servers to ensure no single server becomes overwhelmed. It acts as a traffic director for your application.

How It Works

When users send requests to your application, the load balancer receives them first. Instead of sending all requests to one server, it intelligently distributes them across multiple backend servers using specific algorithms. Each server processes its assigned requests and sends responses back through the load balancer to the users. The load balancer also monitors server health, automatically routing traffic away from failed servers to maintain availability.

Distribution Algorithms

Round Robin

Distributes requests sequentially to each server in rotation. Simple and fair, ideal when servers have similar capacity and requests require similar processing.

Least Connections

Routes traffic to the server with the fewest active connections. Best for applications where request processing time varies significantly.

IP Hash

Uses client's IP address to determine which server receives the request. Ensures the same user always connects to the same server (session persistence).

✓ Key Benefits

Prevents Overload

No single server handles all traffic. Load is distributed evenly, preventing any one server from becoming a bottleneck and ensuring optimal resource utilization.

Increases Availability

If a server fails, the load balancer automatically redirects traffic to healthy servers. Users experience no downtime, maintaining continuous service.

Improves Performance

By distributing load efficiently, response times remain fast even under high traffic. Users get quick responses regardless of overall system load.

Scalability

Easily add or remove servers without disrupting service. The load balancer automatically includes new servers in the rotation, enabling seamless horizontal scaling.



Common Use Cases

Web Applications

Distribute HTTP/HTTPS traffic across web servers

Microservices

Route API requests to service instances

Database Clustering

Distribute read queries across replicas



CACHING - CONCEPTS & EVICTION

Caching stores frequently accessed data in fast, temporary storage to reduce latency and server load. It acts as a speed layer between users and the main data source, dramatically improving application performance.



Cache (Temporary Storage)

A high-speed data storage layer that holds copies of frequently accessed data. Located closer to the user or application, it provides much faster access than the origin server. Examples: Redis, Memcached, CDN, browser cache.



Origin Server (Main Data Source)

The primary database or server that stores the original, authoritative data. Slower to access but contains all data. The cache retrieves data from here during cache misses and stores copies for future requests.



How Caching Works

When a client requests data, the system first checks the cache. This decision point determines whether the response will be fast (cache hit) or require fetching from the origin server (cache miss).

✓ Cache Hit

Data exists in cache. The cached copy is returned immediately to the client. This is the ideal scenario - fast response with no origin server load.

✗ Cache Miss

Data not in cache. System fetches from origin server, returns to client, and stores a copy in cache for future requests. Slower but ensures data freshness.



Cache Eviction Techniques

Cache has limited space. When full, eviction policies determine which data to remove to make room for new entries.

LRU (Least Recently Used)

Removes items that haven't been accessed in the longest time. Assumes recently used data will be used again soon. Best for applications with temporal locality of access patterns.

LFU (Least Frequently Used)

Removes items with the lowest access count. Prioritizes popular data over recent data. Best when some data is consistently more popular than others.

FIFO (First-In, First-Out)

Removes oldest cached item regardless of usage. Simple to implement but doesn't consider access patterns. Best for time-sensitive data with predictable lifecycles.

✓ Key Benefits

Faster Response Times

Cache responds in milliseconds vs. seconds from origin servers. Users experience near-instant load times for frequently accessed content.

Reduced Server Load

Origin servers handle fewer requests, freeing resources for other tasks. Prevents database bottlenecks during traffic spikes.

Lower Network Costs

Reduces data transfer from origin servers. CDNs cache content closer to users, minimizing expensive long-distance data transmission.

Improved Scalability

Handle more users without upgrading origin infrastructure. Cache absorbs traffic increases, allowing applications to scale efficiently.

LATENCY VS THROUGHPUT

Latency and **Throughput** are two fundamental performance metrics in system design. While latency measures how fast a single request completes, throughput measures how many requests can be handled over time. Both are crucial, but often require different optimization strategies.

LATENCY (Time)

The time it takes for a single request to travel from client to server and back. Measures the speed of individual operations.

What It Represents

Think of latency as the time for one car to drive from point A to point B. It's the total duration from when a user clicks a button to when they see the result. Lower latency means faster, more responsive user experiences.

Common Causes of High Latency

Network delays, slow database queries, inefficient code, geographical distance between client and server, and lack of caching all contribute to increased latency.

Measured in: Milliseconds (ms) or Seconds (s)

Goal: Minimize (Lower is better)

Example: 50ms response time for a web page

THROUGHPUT (Volume)

The number of requests a system can handle in a given time period. Measures the capacity and processing power of the system.

What It Represents

Think of throughput as the number of cars passing through a checkpoint in one hour. It measures system capacity regardless of individual request speed. Higher throughput means the system can serve more concurrent users.

Factors Affecting Throughput

Server capacity, parallel processing capabilities, resource availability, and efficient load distribution determine maximum throughput. Bottlenecks anywhere in the system limit overall throughput.

Measured in: Requests per Second (RPS), Transactions per Second (TPS)

Goal: Maximize (Higher is better)

Example: 10,000 requests per second

Relationship & Trade-offs

Independent Yet Related

You can have low latency with low throughput (fast but limited capacity) or high latency with high throughput (slow per request but handles many). Both need optimization.

The Trade-off

Optimizing for one may hurt the other. Adding more concurrent connections increases throughput but may increase latency due to resource contention. Finding balance is key.

Optimization Strategies

Reducing Latency

Use caching, CDNs, optimize database queries, minimize network hops, use faster algorithms, and place servers closer to users geographically.

Balance Both

Use efficient architectures like microservices, implement rate limiting to prevent overload, monitor both metrics continuously, and scale based on actual bottlenecks.

Increasing Throughput

Add more servers (horizontal scaling), use load balancers, implement async processing, optimize resource utilization, and use connection pooling.

Prioritize by Use Case

Real-time apps (gaming, trading) prioritize latency. Batch processing and analytics prioritize throughput. Most apps need both balanced appropriately.



AVAILABILITY & REDUNDANCY

Availability measures how consistently a system remains operational, while **Redundancy** provides backup components to prevent failures. Together, they ensure systems remain reliable and accessible even when individual components fail.

AVAILABILITY (Uptime)

What Is Availability?

Availability is the percentage of time a system is operational and accessible to users. It's calculated as: **Uptime / (Uptime + Downtime)**. Higher availability means users can rely on the system being there when needed.

Availability	Downtime per Year
99% (2 Nines)	3.65 days/year
99.9% (3 Nines)	8.76 hours/year
99.99% (4 Nines)	52.6 minutes/year
99.999% (5 Nines)	5.26 minutes/year

Why It Matters: Each additional "nine" requires exponentially more effort and cost. Critical systems (banking, healthcare) aim for 99.99% or higher, while less critical systems may accept 99.9%.

REDUNDANCY (The Backup)

What Is Redundancy?

Redundancy means having duplicate components (servers, databases, networks) so if one fails, another takes over. It eliminates single points of failure - critical components whose failure would bring down the entire system.

Replication Patterns

Active-Passive

Primary server handles all traffic while secondary waits on standby. If primary fails, secondary activates.

Active-Active

Both servers handle traffic simultaneously, sharing the load. If one fails, the other continues serving all requests.

What Gets Replicated: Servers, databases, load balancers, network connections, power supplies, and even entire data centers across geographic regions.



How Redundancy Improves Availability

When a user sends a request, it goes through a load balancer that routes traffic to available servers. If the primary server fails, the load balancer detects this through health checks and automatically redirects traffic to the secondary server. This **automatic failover** happens in seconds, minimizing downtime. Users experience no interruption because backup systems immediately take over. Without redundancy, server failure means complete downtime until manual intervention. With redundancy, the system self-heals and maintains availability.



Benefits & Trade-offs

✓ Benefits

- **Higher Uptime:** System stays operational despite component failures
- **Fault Tolerance:** Gracefully handles hardware/software failures
- **Seamless Maintenance:** Update components without downtime
- **Disaster Recovery:** Geographic redundancy protects against regional outages

✗ Trade-offs

- **Increased Cost:** Duplicate hardware, software, and infrastructure expenses
- **Complexity:** More components mean harder setup, monitoring, and management
- **Data Consistency:** Keeping replicas synchronized requires careful coordination

▲ CAP THEOREM & USE CASES

The **CAP Theorem** states that in a distributed system, you can only guarantee two out of three properties: **Consistency**, **Availability**, and **Partition Tolerance**. This fundamental constraint shapes how we design distributed databases and systems.

Understanding the Three Components

C - Consistency

All nodes see the same data at the same time. When data is written, all subsequent reads return that value across all nodes. Every client sees the most recent write.

A - Availability

Every request receives a response (success or failure), even if some nodes are down. The system remains operational and accessible, though data may not be the latest.

P - Partition Tolerance

System continues to operate despite network failures that split nodes into isolated groups. The system handles communication breakdowns between nodes gracefully.

⚠ Why You Can Only Pick Two

In a distributed system, network partitions (communication failures between nodes) are inevitable. When a partition occurs, you must choose: guarantee consistency by refusing requests to some nodes (sacrificing availability), or guarantee availability by allowing potentially inconsistent data (sacrificing consistency). You cannot have both during a partition. Since partition tolerance is required for any distributed system, the real choice is between **consistency (CP)** and **availability (AP)**. CA systems are rare in practice because they assume no network partitions, which is unrealistic in distributed environments.

Practical Trade-offs & Use Cases

CA - Consistency & Availability

(Rarely Practical)

Prioritizes consistent data and system availability but cannot tolerate network partitions. Only works in single-location systems without partitioning.

Use Case: Traditional RDBMS in a single location with no partitions. Specific internal applications where network splits are negligible.

CP - Consistency & Partition Tolerance

(Consistency First)

Ensures all nodes always have consistent data, even during network partitions. May refuse requests to maintain accuracy, temporarily reducing availability.

Use Case: Banking systems, financial transactions, inventory management. Any system where accuracy is critical and temporary unavailability is acceptable (e.g., "Transaction cannot be processed right now").

AP - Availability & Partition Tolerance

(Availability First)

System remains available during partitions, but data may be temporarily inconsistent across nodes. Eventual consistency ensures data syncs when partition resolves.

Use Case: Social media feeds, e-commerce shopping carts, DNS, caching systems. Systems where showing content immediately is critical, and temporary inconsistency is acceptable (e.g., "Like count may be slightly off").

🎯 KEY TAKEAWAY

- **Partition Tolerance (P) is mandatory** for distributed systems - network failures will happen
- **Choose between C and A** based on business needs: accuracy vs. continuous access
- **CP systems:** Financial applications where wrong data is worse than temporary downtime
- **AP systems:** Social/content platforms where user experience requires constant access
- **Eventual consistency** is the practical approach for AP systems - data will sync eventually



FORWARD VS REVERSE PROXY

Proxies act as intermediaries providing security, performance, and control. **Forward proxies** represent clients, while **reverse proxies** represent servers.

FORWARD PROXY (Client's Advocate)

Sits between clients and the internet, acting on behalf of clients. Clients route requests through the proxy to access external resources.

How It Works

Clients in a local network send requests to the forward proxy, which forwards them to origin servers. The proxy receives responses and returns them to clients. Origin servers only see the proxy's IP, not individual client IPs.

Use Cases

Content Filtering: Block access to specific websites (e.g., corporate/school networks blocking social media or inappropriate sites).

Anonymity: Hide client IP addresses from origin servers, protecting user privacy online.

Caching: Store frequently accessed resources locally to speed up access and reduce bandwidth usage.

Geo-unblocking: Bypass regional restrictions by routing through proxies in different locations.

REVERSE PROXY (Server's Advocate)

Sits between the internet and backend servers, acting on behalf of servers. Clients send requests to the proxy, which routes them to backend servers.

How It Works

Clients send requests to the reverse proxy. The proxy routes them to backend servers, receives responses, and returns them to clients. Clients never directly interact with backend servers, which remain hidden.

Use Cases

Load Balancing: Distribute incoming traffic across multiple servers to prevent overload and ensure high availability.

Security / WAF: Protect servers from attacks (DDoS, SQL injection) by filtering malicious traffic.

SSL Termination: Handle SSL/TLS encryption/decryption, offloading CPU-intensive work from backend servers.

Caching & Compression: Cache static content and compress responses to improve performance and reduce server load.



Key Differences

Represents:	Clients (hides client identities)	Servers (hides server infrastructure)
Location:	Client network (LAN)	Server network (data center)
Primary Goal:	Control client access, anonymity	Security, load distribution, performance
Configured By:	Clients or network admin	Server/infrastructure admin



Key Takeaway

- **Forward Proxy:** Represents clients - hides who is making requests
- **Reverse Proxy:** Represents servers - hides where requests are processed
- **Both enhance security:** Forward proxies filter outbound; reverse proxies filter inbound traffic
- **Both improve performance:** Through caching, compression, and intelligent routing
- **Common tools:** Nginx, HAProxy, Apache, AWS ALB/ELB



API GATEWAY

An **API Gateway** is a single entry point for all client requests in a microservices architecture. It acts as a reverse proxy, routing requests to appropriate backend services while providing cross-cutting functionality like authentication, rate limiting, and caching.

✗ The Problem

Direct Client-to-Microservice Communication creates complexity and security issues. Clients must know about multiple service endpoints, handle different protocols, implement authentication for each service, and manage failures across many connections. This is complex, insecure, and tightly couples clients to backend architecture.

✓ The Solution

Single Entry Point through an API Gateway simplifies client interactions. Clients make one request to the gateway, which handles routing, security, and protocol translation. This decouples clients from backend services, centralizes cross-cutting concerns, and provides a unified interface regardless of backend complexity.

🔄 How It Works

1

Auth Check

Validates client identity and permissions. Rejects unauthorized requests immediately without reaching backend services.

2

Routing Rule

Determines which backend service should handle the request based on path, headers, or other criteria.

3

Enrich/Transform

Modifies request/response data, aggregates multiple service calls, or converts protocols before returning to client.

⚙️ Key Functions

Routing

Directs requests to the correct backend service based on URL path, request method, or headers.

Authentication & Authorization

Verifies user identity and permissions before allowing access to backend services.

Rate Limiting

Controls traffic flow to prevent abuse, protect services from overload, and ensure fair usage.

Caching

Stores responses to reduce latency, decrease backend load, and improve overall performance.

Protocol Translation

Converts between protocols (HTTP to gRPC, REST to WebSocket) enabling diverse client support.

Request Aggregation

Combines multiple backend calls into a single client request, reducing network overhead.

✓ Key Benefits

Simplified Client Interface

Clients interact with one endpoint instead of tracking multiple service URLs, protocols, and authentication methods.

Improved Security Posture

Centralizes authentication, authorization, and threat protection. Backend services remain hidden from direct exposure.

Centralized Management

Cross-cutting concerns like logging, monitoring, rate limiting handled in one place rather than duplicated across services.

Reduced Backend Complexity

Services focus on business logic without worrying about authentication, rate limiting, or protocol handling.



DISTRIBUTED SYSTEMS

A **distributed system** is a collection of independent computers working together across a network that appears to users as a single, unified system. It breaks down a monolithic application into multiple nodes that collaborate to achieve common goals.

The Problem: Monolithic Systems

Single Point of Failure: A monolithic system runs on one server. When that server crashes or reaches capacity, the entire application goes down. Resources are limited to one machine, making it hard to scale. All components (application, database, services) depend on one infrastructure, creating a critical vulnerability.

The Solution: Distribution

Multiple Independent Nodes: Distributed systems spread the workload across many computers connected by a network. Load balancers distribute traffic, multiple application servers handle requests in parallel, and databases can be replicated. If one node fails, others continue operating. The system scales by adding more nodes.

✓ Key Characteristics

- ✓ **Scalability:** Horizontal scaling by adding more nodes easily. Handle increased load by distributing work across additional machines without architectural changes.
- ✓ **Reliability:** Fault tolerance through redundancy. No single point of failure - if one node fails, others continue serving requests seamlessly.
- ✓ **Performance:** Parallel processing enables faster execution. Multiple nodes work simultaneously on different parts of the workload, reducing overall processing time.
- ✓ **Resource Sharing:** Efficient use of hardware across multiple machines. Computing power, storage, and bandwidth are pooled and allocated as needed.

⚠ Key Challenges

- ⚠ **Complexity:** Harder to design, build, and manage than monolithic systems. Requires understanding of distributed algorithms, deployment strategies, and debugging across multiple machines.
- ⚠ **Network Issues:** Latency between nodes adds delays. Network partitions can split the system. Failures in communication require retry logic and timeout handling.
- ⚠ **Data Consistency:** Keeping data synchronized across nodes is difficult. CAP Theorem forces trade-offs between consistency, availability, and partition tolerance.
- ⚠ **Coordination:** Synchronization between nodes is complex. Achieving consensus (agreement on state) requires sophisticated algorithms like Paxos or Raft.

🎯 Key Takeaway

Distributed systems offer **massive scale and reliability** by spreading workload across many machines, eliminating single points of failure, and enabling parallel processing. However, they introduce **significant complexity** in coordination, consistency, and failure handling. Network latency, data synchronization, and consensus mechanisms require careful design. Choose distributed architecture when scalability and fault tolerance requirements justify the added complexity.



INDEXING IN DATABASES

Database indexes are data structures that improve query performance by creating shortcuts to data. Like a book's index, they allow quick lookup without scanning every row. They trade storage space and write speed for dramatically faster reads.

What Are Indexes?

An index is a separate data structure (commonly B-Tree or Hash Table) that stores sorted copies of selected columns along with pointers to the actual data rows. When you query indexed columns, the database uses the index to locate rows quickly instead of scanning the entire table. For example, an index on a "user_id" column allows `SELECT * FROM users WHERE user_id = 123` to find the row in milliseconds rather than seconds.

✓ Usefulness

Faster Query Performance: Dramatically speeds up data retrieval for `SELECT` queries with `WHERE` clauses. Queries that would scan millions of rows can return results in milliseconds.

Efficient Sorting: Optimizes `ORDER BY` and `GROUP BY` operations. Since indexes store data in sorted order, sorting operations become nearly instant.

Improved Joins: Accelerates table linking in `JOIN` operations. When joining tables on indexed columns, the database can match rows efficiently without full table scans.

Enforced Uniqueness: Guarantees unique values through `PRIMARY KEY` and `UNIQUE` constraints. The database uses indexes to quickly check for duplicates during inserts.

Optimized Aggregation: Speeds up aggregate functions like `SUM`, `COUNT`, `AVG`, `MIN`, `MAX`. Indexes allow the database to compute results without processing every row.

⚠ Challenges

Increased Storage Space: Each index creates an additional data structure consuming disk space. Multiple indexes on large tables can significantly increase storage requirements.

Slower Write Operations: `INSERT`, `UPDATE`, and `DELETE` operations become slower because the database must update both the table and all its indexes.

Maintenance Overhead: Requires regular monitoring, rebuilding, and reorganizing. Neglected indexes can become inefficient over time.

Complexity in Design: Choosing the right index type (B-Tree, Hash, Bitmap) and columns requires understanding query patterns and database internals.

Index Fragmentation: As data changes, indexes become fragmented, degrading performance. Regular defragmentation or rebuilding is necessary to maintain efficiency.



Best Practices & Key Takeaway

- **Balance Read vs. Write Needs:** Read-heavy applications benefit greatly from indexes. Write-heavy systems may need fewer indexes to maintain performance.
- **Index Selectively:** Analyze actual query patterns before creating indexes. Index columns frequently used in `WHERE`, `JOIN`, `ORDER BY`, and `GROUP BY` clauses.
- **Monitor & Maintain Regularly:** Track index usage, remove unused indexes, and rebuild fragmented ones. Use database monitoring tools to identify missing or inefficient indexes.
- **Consider Composite Indexes:** For queries filtering on multiple columns, composite indexes (multi-column) often outperform multiple single-column indexes.
- **Avoid Over-Indexing:** Too many indexes hurt write performance and waste storage. Every index should serve a clear, measurable purpose.



DATA PARTITIONING

Data partitioning (also called sharding) divides large databases into smaller, independent parts. Each partition operates as a separate unit, enabling horizontal scaling and improved performance while maintaining logical unity.

The Problem & Solution

✗ Monolithic Database

As data grows, a single database becomes a bottleneck. Queries slow down as they scan millions or billions of rows. Storage limits are reached. The database becomes a single point of failure - if it crashes, the entire application goes down. Vertical scaling (bigger hardware) has limits and becomes expensive.

✓ Partitioning Solution

Divide data into independent, distinct parts distributed across multiple servers. Each partition contains a subset of data and operates independently. Queries run on smaller datasets, multiple partitions can be queried in parallel, and the system scales horizontally by adding more partitions.

Common Partitioning Strategies

Horizontal Partitioning (Sharding)

Splits data by rows - different rows go to different partitions. Each partition has the same schema but contains different records. Commonly uses range-based (Users 1-1000 in Partition 1) or hash-based (hash user_id to determine partition) distribution.

Best for: Scaling data volume. User databases (partition by user_id), time-series data (partition by date), geographic data (partition by region).

Vertical Partitioning

Splits data by columns - different columns go to different partitions. Separates frequently accessed columns from rarely accessed ones, or large BLOBs from small metadata. Each partition has different columns but same row keys.

Best for: Optimizing specific queries. Store frequently accessed columns (ID, Name) separately from large, rarely used data (PhotoBlob, Biography).

✓ Benefits

Scalability: Scale out easily by adding more partitions. Distribute data across multiple servers without architectural changes.

Performance: Faster queries on smaller datasets. Each partition processes only its subset of data, reducing query time significantly.

Availability: Failure in one partition doesn't kill the entire system. Other partitions continue serving requests independently.

Manageability: Easier maintenance and backups. Can upgrade, backup, or maintain partitions individually without full system downtime.

⚠ Challenges

Complexity: More moving parts to manage. Requires partition routing logic, monitoring multiple databases, and coordinating operations.

Cross-Partition Joins: Joining data across partitions is difficult and slow. Queries spanning partitions require network calls and data aggregation.

Rebalancing Data: Moving data between partitions can be complex. As partitions grow unevenly, redistributing data requires careful planning and execution.

Key Takeaway: A fundamental technique for building large-scale systems, trading some complexity for significant gains in scalability and performance. Choose partitioning strategy based on query patterns and growth expectations.



SHARDING

Sharding is a horizontal partitioning technique that splits a large database into smaller, independent pieces called shards. Each shard is a separate database that holds a portion of the total data, enabling massive scalability.

The Problem: Overloaded Database

A single database becomes a bottleneck as data and traffic grow. It struggles with massive datasets, queries slow to a crawl, and the system hits hardware limits. One failure brings down the entire application - a critical single point of failure that's expensive to scale vertically.

The Solution: Sharding

Break data into smaller, manageable parts (shards) distributed across multiple servers. Each shard operates independently with its own subset of data. Queries run faster on smaller datasets, the system scales horizontally by adding shards, and failures are isolated to individual shards.

How It Works: Routing Layer

When a client sends a query (e.g., WHERE ID=123), it goes to a **routing layer** or **router**. The router uses a **shard key** (like user ID) to determine which shard contains the data. For example, using $ID \% 3$, if ID=123, then $123 \% 3 = 0$, directing the query to Shard 0. The router maintains the mapping logic and directs each query to the correct shard server, ensuring data is accessed efficiently without scanning all shards.

Sharding Strategies

Key-based (Hash) Sharding

Uses a hash function on the shard key to determine placement: $\text{Hash}(\text{Key}) \% N$. The hash distributes data uniformly across N shards.

Pros: Uniform distribution

Cons: Hard to reshard when adding/removing shards

Range-based Sharding

Assigns data to shards based on ranges of the shard key. Example: A-M goes to Shard 1, N-Z goes to Shard 2.

Pros: Good for range queries, intuitive

Cons: Can create hotspots if ranges are unbalanced

Directory-based Sharding

Maintains a lookup table (directory) that maps each key to its shard. The system queries the directory to find which shard holds the data.

Pros: Flexible, easy to rebalance

Cons: Adds lookup overhead, directory can become bottleneck

Benefits & Challenges

✓ Benefits

- **Horizontal Scalability:** Add more shards to handle growth without limits
- **Improved Performance:** Queries run on smaller datasets, returning results faster
- **Fault Isolation:** Failure in one shard doesn't affect others, maintaining partial availability

⚠ Challenges

- **Increased Complexity:** Managing multiple databases, routing logic, and monitoring
- **Cross-Shard Queries:** Queries spanning shards are slow and complex
- **Data Rebalancing:** Redistributing data when adding/removing shards is difficult



DESIGN A RATE LIMITER

A **rate limiter** controls the rate of traffic sent by clients or services, preventing abuse and ensuring system stability. It limits requests based on rules like requests per second (RPS) or total quota within a time window.

Key Concepts

Requests per Second (RPS): Maximum number of requests allowed over time (e.g., 100 requests/second per user).

Quota: Total allowed requests in a specific period (e.g., 10,000 requests per day).

Window Size: Timeframe for counting requests (e.g., 1 minute, 1 hour).

Throttling: Slowing down requests instead of blocking them completely, maintaining a steady flow.

How It Works

When a client sends a request, it passes through the rate limiter before reaching backend services. The limiter checks rules stored in a fast cache (Redis) - tracking request counts or token availability. If the limit is exceeded, the request is blocked with a **429 Too Many Requests** response and a **Retry-After** header indicating when to retry. Allowed requests proceed to the backend service for processing.

Rate Limiting Algorithms

Token Bucket

Tokens are added to a bucket at a fixed rate. Each request consumes one token. If no tokens are available, the request is rejected. Allows burst traffic up to bucket capacity.

Leaky Bucket

Requests enter a bucket and are processed at a constant rate (leak out). If the bucket fills up, new requests are dropped. Enforces steady output rate regardless of input.

Fixed Window

Counts requests in fixed time windows (e.g., 0:00-0:59, 1:00-1:59). Simple but can allow twice the limit at window boundaries.

Sliding Window Log

Tracks timestamps of all requests. Counts requests within a rolling time window. Accurate but memory-intensive for high traffic.

Sliding Window Counter

Hybrid approach combining fixed and sliding windows. Uses weighted calculation to estimate requests in the current window. More accurate than fixed window, more efficient than log.

✓ Benefits

Prevent Abuse/DoS Attacks

Stops malicious users from overwhelming the system with excessive requests, protecting against denial-of-service attacks.

Ensure Fairness Among Users

Prevents any single user from monopolizing resources, ensuring all users get fair access to the service.

Protect Backend Services

Shields databases and application servers from overload, maintaining system stability and preventing crashes.

Manage Costs & Resources

Controls infrastructure costs by limiting API usage, especially important for services with per-request pricing (third-party APIs).



SQL VS NOSQL DATABASES

SQL (Relational) and **NoSQL (Non-Relational)** databases represent two fundamentally different approaches to data storage. SQL uses structured tables with fixed schemas, while NoSQL offers flexible, schema-less models for diverse data types.

SQL (Relational) - Structured & Rigid

Data Structure: Tables with rows and columns (relational model). Data organized in predefined table structures with relationships enforced through foreign keys.

Schema: Fixed, pre-defined schema. Changes to structure are hard and require migrations. Every record must follow the same structure.

Scaling: Vertical scaling (scale up with bigger machines). Adding more powerful hardware to single server. Horizontal scaling is difficult.

Query Language: Structured Query Language (SQL). Powerful complex queries with JOINS, aggregations, and sophisticated filtering.

Examples: MySQL, PostgreSQL, Oracle

NoSQL (Non-Relational) - Flexible & Dynamic

Data Structure: Document, Key-Value, Graph, or Column-family models. Diverse structures for different use cases, no fixed table format.

Schema: Dynamic, schema-less. Easy to change structure. Each record can have different fields without migration.

Scaling: Horizontal scaling (scale out with more machines). Designed for distributed systems. Add servers easily to handle growth.

Query Language: API-based with focused methods. Simpler queries, less powerful joins. Each database has its own query approach.

Examples: MongoDB, Redis, Cassandra, Neo4j

When to Use Each

Choose SQL For:

- Complex queries with multiple table JOINS
- ACID transactions (banking, financial systems)
- Structured data with clear relationships
- Data integrity and consistency requirements
- Mature, well-understood query optimization

Choose NoSQL For:

- Rapid development with changing requirements
- Massive scale and high-velocity data
- Unstructured or semi-structured data
- Horizontal scalability needs
- High read/write throughput (real-time apps)

 **Key Takeaway:** Choose based on data structure, scalability needs, and query complexity. SQL for complex queries & transactions; NoSQL for flexibility, speed & massive scale. Many modern systems use both (polyglot persistence).



ACID PROPERTIES

ACID is a set of properties that guarantee reliable processing of database transactions. These guarantees ensure data integrity and consistency even when system failures occur during transaction execution.

The Problem Without ACID

Consider a money transfer from Account A to Account B. Without ACID guarantees, the database might deduct money from Account A, then crash before adding it to Account B. The result: Account A loses money, Account B never receives it, and the data becomes inconsistent. ACID properties prevent such scenarios by ensuring transactions either complete fully or not at all, maintaining data integrity throughout.

The Four ACID Properties

A - Atomicity

"All or Nothing"

A transaction is treated as a single, indivisible unit. Either all operations within the transaction succeed (commit), or none of them do (rollback). If the system crashes mid-transaction, all partial changes are reverted. In the money transfer example, both the deduction from A and addition to B happen together, or neither happens.

C - Consistency

"Valid State to Valid State"

Transactions move the database from one valid state to another, preserving all defined rules and constraints. For example, if a rule states "balance must be ≥ 0 ", the database ensures this rule holds before and after every transaction. No transaction can violate data integrity constraints.

I - Isolation

"Concurrent but Independent"

Multiple transactions can execute concurrently, but each operates as if it's the only transaction running. Transactions don't interfere with each other - one transaction's intermediate changes are invisible to others until committed. This prevents race conditions and ensures data accuracy with concurrent access.

D - Durability

"Once Committed, It's Permanent"

Once a transaction is committed, its changes are permanent and survive system failures. Committed data is written to stable storage (disk/SSD) and persists even if the system crashes immediately after. Database recovery mechanisms ensure committed transactions are never lost.

✓ Benefits

Data Integrity: Guarantees trustworthy, consistent data. No partial updates or corrupted states.

Reliability: Gracefully handles failures with automatic rollback. System recovers to consistent state.

Simplified Logic: Developers write less error-handling code. Database manages transaction safety automatically.

⚠ Challenges

Performance Overhead: Slower operations due to locking mechanisms, logging, and synchronization required for ACID guarantees.

Scalability Issues: Harder to scale horizontally across distributed systems. Strong consistency limits distribution.

Complexity: Implementation requires sophisticated mechanisms for transaction management and recovery.



Key Takeaway: ACID provides strong data guarantees, essential for critical systems (e.g., banking, financial transactions), but comes with performance trade-offs compared to BASE (NoSQL). Choose ACID when correctness is paramount over speed.



BASE PROPERTIES

BASE is an alternative consistency model for NoSQL databases, trading immediate consistency for higher availability and scalability. It's designed for large-scale, distributed systems where ACID's strictness becomes a limitation.

The Need for Flexibility

Problem: Rigid ACID System

ACID guarantees strict consistency but struggles with massive, distributed data at high velocity. Scaling horizontally becomes difficult, and the system can become slow and stressed under heavy load. ACID's strictness limits scale and availability.

Solution: BASE Model

BASE relaxes consistency requirements, prioritizing availability and scalability over immediate consistency. This allows distributed systems to handle massive data volumes and continue operating during network partitions.

The Three BASE Properties

BA - Basically Available

"Always Responsive, Mostly"

The system guarantees availability for most requests, but not all data may be reachable at all times. Rather than blocking requests during failures, the system continues responding - perhaps with stale data or partial results. The focus is on keeping the system operational even if some nodes are down.

S - Soft State

"State Can Change, Even Without Input"

The state of the system may change over time due to internal processes like replication and synchronization, not just user updates. Data might be in flux as it propagates across distributed nodes. The system doesn't guarantee immediate consistency - changes flow through the system gradually.

E - Eventual Consistency

"Consistent, Eventually"

The system will become consistent over time, given that updates stop. All nodes will eventually see the same data, but there may be a delay. During this window, different nodes might return different values. This temporary inconsistency is the trade-off for availability and performance.

✓ Benefits (vs. ACID)

High Scalability: Scale out easily by adding more nodes. Distributed architecture handles massive data volumes.

High Availability: System remains resilient to failures. Continues operating even when some nodes are down.

Flexible Schema: Adapts to changing requirements easily. No rigid schema migrations needed for structure changes.

Better Performance: Lower latency due to fewer locks and less coordination overhead between nodes.

⚠ Challenges (vs. ACID)

Data Inconsistency: Temporary versions of data exist across nodes. Users might see stale or conflicting information.

Complex App Logic: Application must handle conflicts and inconsistencies. Developers write more complex code to manage eventual consistency.

No Strong Guarantees: Trade-off for availability. Cannot guarantee immediate consistency or transaction isolation.

Not for Critical Transactions: Unsuitable for systems requiring strict consistency (e.g., banking, financial transactions).

🔑 Key Takeaway: BASE trades immediate consistency for higher availability and scalability, making it suitable for large-scale, distributed web applications and big data (AP in CAP). Choose BASE over ACID when scale and speed matter more than instant consistency.



SERVERLESS COMPUTING

Serverless computing is a cloud execution model where developers write and deploy code without managing servers. The cloud provider handles all infrastructure, automatically scaling resources and charging only for actual execution time.

The Concept: No Servers to Manage

Despite the name, servers still exist - but they're completely abstracted away. Developers don't provision, configure, patch, scale, or maintain servers. The cloud provider handles all infrastructure concerns including security updates, OS patches, capacity planning, and hardware maintenance. This allows developers to focus entirely on writing business logic and application code.

Key Insight: You write code, the cloud provider runs it. No server administration, no infrastructure management - just pure application development.

How It Works: Event-Driven Architecture

Serverless operates on an **event-driven model** using **Functions as a Service (FaaS)**. Functions are small, single-purpose pieces of code that execute in response to events or triggers.

1. Trigger/Event

HTTP request, file upload, database change, scheduled time, or message queue event

2. Function Execution

Cloud provider spins up a container, runs your code, then tears it down

3. Action/Output

Update database, send email, return response - then function terminates

Pay-per-use model: You're charged only for execution time and resources consumed (measured in milliseconds and memory). No charges when code isn't running. Examples: AWS Lambda, Azure Functions, Google Cloud Functions.

✓ Benefits

No Server Management: Operations-free - no patching, provisioning, or maintenance. Cloud provider handles all infrastructure.

Auto-scaling: Instant, elastic scaling. Automatically handles one request or millions without manual intervention.

Cost Efficiency: Pay-as-you-go pricing. Only charged for actual execution time, not idle server time. Ideal for variable workloads.

Faster Time to Market: Focus on code, not infrastructure. Deploy quickly without managing servers, networking, or capacity planning.

⚠ Challenges

Cold Starts: First request after idle period has latency while function initializes. Can add 100ms-several seconds delay.

Vendor Lock-in: Hard to switch providers. Each platform has unique APIs, configurations, and deployment processes.

Debugging Complexity: Distributed, ephemeral nature makes debugging difficult. Functions exist briefly, making troubleshooting harder.

Execution Limits: Time and memory constraints. Most providers limit execution to 5-15 minutes with specific memory caps.

Key Takeaway: Focus on writing code, not managing infrastructure. Ideal for event-driven, scalable applications with variable workloads. Best for microservices, APIs, data processing, and automation tasks where infrastructure management overhead isn't justified.



STATELESS VS STATEFUL

Stateless and **Stateful** architectures differ in how they handle client data across requests. Stateless servers treat each request independently with no memory, while stateful servers retain client information across multiple interactions.

STATELESS (No Memory)

How it works: Each request is completely independent. The server doesn't save any client data between requests. Any server can handle any request since no session information is stored. The client must send all necessary data with every request.

✓ Pros

- **Scalability:** Easy to add servers - any server can handle any request
- **Fault Tolerance:** If a server fails, another takes over seamlessly

✗ Cons

- **Data Overhead:** Client must send all data with every request
- **Complexity:** State must be managed externally (DB, cache)

Best for: REST APIs, CDN, Microservices (often)

STATEFUL (Has Memory)

How it works: Server retains client data (state) across multiple requests. Uses session IDs to identify returning clients. Load balancers use "sticky sessions" to route clients to the same server, ensuring access to their session data.

✓ Pros

- **Simpler Logic:** State available locally on the server
- **Performance:** Less data transfer per request

✗ Cons

- **Scalability Issues:** Sticky sessions make scaling harder
- **Fault Tolerance:** If server fails, session data is lost

Best for: Online Gaming, Databases, FTP, Real-time apps



Key Differences

Aspect	Stateless	Stateful
Focus:	Independent requests	Client session continuity
Data Handling:	Client sends all data each time	Server keeps client data in memory
Coupling:	Loose (tight between client & data)	Tight (client tied to specific server)
Scalability:	High - easy to add servers	Limited - harder due to sticky sessions

🎯 **Decision Guide:** Choose **stateless** for scalable, distributed systems with independent requests (REST APIs, microservices). Choose **stateful** for applications requiring persistent connections and real-time interaction (gaming, streaming, live collaboration). Modern hybrid approaches often use stateless architecture with external state storage (Redis, databases).



MONOLITHIC VS MICROSERVICES

Monolithic and **Microservices** represent two fundamentally different approaches to structuring applications. Monolithic builds everything as one unified block, while microservices decomposes functionality into small, independent services.

MONOLITHIC (The Big Block)

What it is: A single, unified application where all components (UI, business logic, data access) are tightly bundled together. Everything runs as one process, shares the same codebase, and deploys as a single unit. One file to deploy, one database, one technology stack.

✓ Pros

- **Simplicity:** Easier to develop and test initially
- **Simple Deployment:** One file to deploy

✗ Cons

- **Limited Scalability:** Scale entire app, even if only one part needs it
- **Complexity over Time:** Becomes hard to understand and maintain
- **Single Point of Failure:** One bug can crash everything
- **Technology Lock-in:** Hard to change tech stack

MICROSERVICES (Small, Connected)

What it is: Collection of small, autonomous services where each handles a specific business function. Services communicate via APIs (REST, messaging), have their own databases, and can be deployed independently. Loose coupling, high cohesion.

✓ Pros

- **High Scalability:** Scale individual services independently
- **Fault Isolation:** Failure in one doesn't crash the whole system
- **Technology Diversity:** Use different stacks for different services
- **Independent Deployment:** Faster, more frequent updates

✗ Cons

- **Increased Complexity:** Distributed system challenges
- **Data Consistency:** Harder to maintain across services
- **Network Latency:** Communication between services adds time



Key Differences & Trade-offs

Aspect	Monolithic	Microservices
Scalability:	Limited - scale entire app	High - scale services independently
Development:	Simpler initially	Complex (distributed challenges)
Complexity:	Lower initially, grows over time	Higher operationally from start
Data Management:	Centralized (simpler)	Decentralized (complex)

🎯 **Decision Guide:** Start with **monolithic** for MVPs, small teams, or simple applications. Move to **microservices** when scaling demands, team size, or complexity justify the operational overhead. Many successful companies start monolithic and gradually extract services as they grow. Consider team expertise and infrastructure capabilities before choosing.



AUTOSCALING

Autoscaling automatically adjusts computing resources to match demand in real-time. It adds servers during traffic spikes and removes them during quiet periods, ensuring performance while optimizing costs.

The Concept: Dynamic Resource Adjustment

The Problem with Fixed Servers: Static infrastructure either wastes money (over-provisioned for peak load) or provides poor performance (under-provisioned for spikes). You pay for unused capacity during low traffic and suffer overload during high traffic.

The Solution: Autoscaling creates an Auto Scaling Group (ASG) - a pool of servers that automatically expands and contracts based on demand. During traffic spikes, new instances launch within minutes. During low traffic, unnecessary instances terminate to save costs. Resources match actual need at any given moment.

How It Works: The Scaling Loop

Autoscaling operates in a continuous monitoring and adjustment cycle:

1

Monitor

Track metrics like CPU > 80%, Memory > 70% across server group

2

Alarm

Trigger alarm when threshold is breached

3

Policy

Scaling policy defines action (e.g., add 2 instances)

4

Execute

Launch or terminate servers via cloud API

5

Update

Register/deregister with load balancer

✓ Key Benefits

Cost Savings: Pay only for what you use. Automatically reduces capacity during low demand, eliminating wasted resources.

High Availability: Handles traffic spikes and server failures automatically. Replaces unhealthy instances without manual intervention.

Consistent Performance: Prevents overload by adding capacity before users experience slowdowns. Maintains response times under varying load.

Reduced Management: Automated operation eliminates manual capacity planning. No need to predict traffic or provision ahead of time.



Scaling Strategies

Dynamic/Reactive: Scale based on real-time metrics (CPU, memory, requests per second). Responds to actual demand as it happens.

Predictive: Scale based on historical patterns and machine learning. Anticipates demand before it occurs using past traffic data.

Scheduled: Scale at specific times based on known patterns (e.g., scale up before a sale at 9 AM, scale down at midnight).



Key Takeaway: Automatically adjusts computing resources to match demand, ensuring performance and cost-efficiency in dynamic environments. Essential for modern cloud applications with variable traffic patterns. Eliminates the traditional trade-off between over-provisioning (waste) and under-provisioning (poor performance).



STRONG VS EVENTUAL CONSISTENCY

Consistency models define how and when data changes become visible across distributed systems. **Strong consistency** guarantees immediate, synchronized updates across all nodes, while **eventual consistency** allows temporary differences that resolve over time.

STRONG CONSISTENCY (Strict & Immediate)

The Guarantee: All nodes see the same data at all times. After a write completes, every subsequent read returns that value across all replicas immediately. No node ever sees stale data - perfect synchronization is maintained at all times.

Data View: Same view at same time - every node shows identical data instantly after any update.

Update Behavior: Updates are immediate and block until acknowledged by all replicas. The write operation doesn't complete until all copies are synchronized.

Latency: Higher - waiting for synchronization across all nodes adds delay. Each write must coordinate across the system.

Availability: Lower - prioritizes consistency over uptime (CP in CAP theorem). System may block or reject requests during network partitions.

Implementation: Complex to implement and manage. Requires distributed transactions, two-phase commits, or consensus algorithms.

Best For: Critical data, financial transactions, banking systems, stock market tickers, systems requiring absolute accuracy. Examples: RDBMS with ACID properties.

EVENTUAL CONSISTENCY (Flexible & Delayed)

The Guarantee: Nodes may see different data versions temporarily, but the system becomes consistent over time given that updates stop. All replicas will eventually converge to the same state, but there's an inconsistency window.

Data View: Different views temporarily - nodes may see different data versions during the propagation period.

Update Behavior: Updates propagate asynchronously. Writes complete immediately without waiting for all replicas to acknowledge. Convergence happens in the background.

Latency: Lower - non-blocking, fast responses. Writes return immediately without coordination overhead.

Availability: Higher - prioritizes uptime over immediate consistency (AP in CAP theorem). System remains available during network partitions.

Implementation: Simpler to implement and scale. Uses gossip protocols, anti-entropy mechanisms, or vector clocks for eventual convergence.

Best For: High-traffic web apps, content delivery, social media feeds, DNS, systems where temporary inconsistency is acceptable. Examples: NoSQL databases with BASE properties, CDN.

Key Trade-offs

Strong Consistency Chooses:

- Correctness over speed
- Accuracy over availability
- Simplicity for developers (predictable reads)
- Costs: Performance and scalability

Eventual Consistency Chooses:

- Speed over immediate correctness
- Availability over instant accuracy
- Scalability and performance
- Costs: Application complexity (handling conflicts)

🎯 **Decision Guide:** Choose based on the trade-off between immediate data accuracy (Strong) and system performance/availability (Eventual). Use **strong consistency** for critical data where correctness is paramount (banking, financial systems). Use **eventual consistency** for speed and scale where temporary inconsistency is acceptable (social media, content delivery, caching). The choice fundamentally depends on whether your application can tolerate stale reads.



PUB/SUB MODEL

The **Publish-Subscribe (Pub/Sub) model** is a messaging pattern where senders (publishers) and receivers (subscribers) communicate indirectly through topics. Publishers don't know who receives their messages, and subscribers don't know who sends them - complete decoupling.

Concepts & Roles

Publisher (Sender): Creates and sends messages to topics. Doesn't know or care who subscribes. Simply publishes information to a specific topic/channel and moves on. Example: A weather service publishing temperature updates.

Subscriber (Receiver): Expresses interest in specific topics and receives relevant messages. Doesn't know who published them. Can subscribe to multiple topics simultaneously. Example: A mobile app subscribing to weather updates for specific cities.

Topic/Channel (Intermediary): Named channel or category where messages are published and distributed. Acts as the middleman connecting publishers to subscribers. Topics organize messages by subject (e.g., "Sports", "Tech", "News").

Message Broker: Infrastructure component (like Kafka, RabbitMQ, Redis) that manages topics, routes messages, and handles delivery. Ensures messages reach all relevant subscribers.

Key Principle - Decoupling: Publishers and subscribers are completely independent. They can be added, removed, or modified without affecting each other. This enables independent scaling and evolution.

How It Works: The Flow

1

Publish

Publisher creates a message and sends it to a specific topic through the message broker. Message includes topic identifier and data payload.

2

Route (by Topic)

Message broker receives the message and identifies all subscribers interested in that topic. Maintains subscription registry internally.

3

Deliver

Broker delivers copies of the message to all relevant subscribers asynchronously. Each subscriber receives their own copy independently.

Example: Publisher P1 sends "Msg A" to topic "Sports". The broker routes it to subscribers S1 and S2 who both subscribed to "Sports". Publisher P2 sends "Msg B" to "Tech", reaching subscriber S3. No publisher knows which subscribers exist.

Benefits & Use Cases

✓ Key Benefits

Loose Coupling: Publishers and subscribers are independent. Can scale and evolve separately without breaking the system.

Scalability: Easily handles many publishers and subscribers. Add more without changing existing code.

Asynchronous: Non-blocking communication. Publishers continue immediately without waiting for subscribers to process messages.

Flexibility: Dynamic subscriptions - subscribers can join or leave at any time. Topics can be created on demand.

💡 Common Use Cases

Event Notifications: Social media updates, breaking news alerts, system event broadcasting.

Chat Applications: Group messages where multiple users subscribe to conversation topics.

IoT Data Feeds: Sensor readings published to topics, consumed by multiple analytics systems.

Distributed Logging & Analytics: Applications publish log events consumed by multiple monitoring tools.



Key Takeaway: Decoupled, asynchronous communication pattern for building scalable and flexible distributed systems. Publishers and subscribers operate independently through topic-based routing, enabling systems to grow and evolve without tight dependencies. Essential for event-driven architectures and microservices.

CONSISTENT HASHING

Consistent hashing is a distributed hashing technique that minimizes data movement when servers are added or removed. Unlike traditional hashing, it maps both data and servers onto a hash ring, ensuring only a small fraction of keys need remapping during topology changes.

The Problem with Traditional Hashing

Traditional approach: Using $\text{Hash}(\text{Key}) \% N$ where N is the number of servers. When you have 4 servers, a key might go to server $\text{Hash}(\text{Key}) \% 4$. This works fine with a fixed number of servers.

The problem: When you add or remove a server (changing N), the modulo operation changes for almost ALL keys. If you go from 4 to 5 servers, most keys will hash to different servers, requiring massive data remapping. For example, a key that was on server 2 might now belong to server 4. This causes enormous data movement and cache invalidation - potentially requiring redistribution of 80-90% of all data.

How Consistent Hashing Works: The Ring

The Hash Ring: Imagine a circular ring with values from 0 to $2^{32}-1$. Both servers and data keys are hashed and placed at specific positions on this ring.

Server placement: Each server is hashed using its identifier: $\text{Hash}(\text{ServerID})$ determines its position on the ring. For example, $\text{Hash}(S1)$, $\text{Hash}(S2)$, etc. place servers at different points around the ring.

Data placement: Each data key is also hashed: $\text{Hash}(\text{DataKey})$ determines where it sits on the ring. To find which server stores a key, move clockwise from the key's position until you encounter the next server. That server owns the key.

Adding a server: When server $S5$ is added between $S2$ and $S3$, only keys that fall between $S2$ and $S5$ need to move to $S5$. All other keys remain on their current servers. This is typically only 10-20% of total data (roughly $1/N$ of the keys, where N is the number of servers).

Removing a server: When a server is removed, only its keys move to the next clockwise server. Again, minimal data movement compared to traditional hashing.

Virtual Nodes: Balancing the Load

The problem: Random hash placement might create uneven distribution. One server might get far more keys than others due to how servers happen to be distributed on the ring.

The solution: Each physical server maps to multiple points on the ring called "virtual nodes." Server A might map to positions $A1$, $A2$, $A3$, etc. Server B maps to $B1$, $B2$, $B3$, etc. This creates many more points on the ring, resulting in much better data distribution. If one physical server has 150 virtual nodes, and there are 5 servers, that's 750 points on the ring, ensuring very even distribution. Virtual nodes also help with load balancing - more powerful servers can have more virtual nodes to handle proportionally more data.

Benefits & Use Cases

✓ Key Benefits

Minimal Rebalancing: Only $\sim 1/N$ of data moves when adding/removing servers, not 80-90% like traditional hashing.

Seamless Scalability: Add or remove nodes without major disruption. System continues operating smoothly.

Hotspot Mitigation: Virtual nodes ensure even data distribution, preventing any single server from being overloaded.

💡 Common Use Cases

Distributed Caching: Amazon Dynamo, Memcached, Redis clusters use consistent hashing to distribute cache entries.

Load Balancing: Distributing requests across multiple servers with minimal disruption during scale changes.

Data Partitioning: Distributed databases use it to partition data across nodes efficiently.

 **Key Takeaway:** Consistent hashing minimizes data movement during scale changes by mapping both servers and data onto a circular hash ring. When servers are added or removed, only a small fraction of keys need remapping, making it essential for building scalable distributed systems like caches, databases, and load balancers.



MESSAGE QUEUES & CDN

Message Queues enable asynchronous communication between services, while **CDNs** cache content globally to accelerate delivery. Both are essential building blocks that improve scalability and performance in different ways.

MESSAGE QUEUES (Async Messaging)

A message queue enables asynchronous communication between services using a producer-consumer pattern. The producer sends messages to the queue without waiting for processing, and consumers retrieve and process messages independently.

How it works: Producer (sending service) publishes messages to the queue. Messages wait in line. Consumer (receiving service) pulls messages when ready and processes them. The queue ensures delivery even if the consumer is temporarily unavailable.

Key Use Cases

Asynchronous Processing: Offload time-consuming tasks like email sending, video transcoding, or report generation. User gets immediate response while work happens in background.

Decoupling: Services don't need to know about each other. Producer and consumer can be developed, deployed, and scaled independently without tight dependencies.

Reliability/Buffering: Handle traffic spikes by absorbing burst requests. Ensures data isn't lost if a consumer is down - messages persist in the queue until processed.

Examples: RabbitMQ, Apache Kafka, Amazon SQS, Redis Streams

CDN (Global Caching)

A Content Delivery Network is a geographically distributed network of servers (edge servers) that cache and deliver content from locations close to end users. It reduces latency by serving content from the nearest edge server rather than the origin server.

How it works: Static content (images, videos, CSS, JavaScript) is cached on edge servers worldwide. When a user requests content, it's served from the nearest edge server. If not cached, the CDN fetches it from the origin server, caches it, then delivers it.

Key Use Cases

Faster Content Delivery: Deliver static assets (images, videos, CSS, JavaScript) quickly to end-users from nearby edge servers. Reduces latency dramatically for global users.

Reduced Server Load: Offload traffic from the origin server by serving cached content. Prevents overload and allows origin to focus on dynamic content and business logic.

Improved Availability: Handle high traffic volumes through distributed infrastructure. Mitigate DDoS attacks by absorbing malicious traffic across many edge servers.

Examples: Cloudflare, Akamai, AWS CloudFront, Fastly

Key Takeaway

Message Queues (Backend)

Decouple services and handle traffic spikes through asynchronous messaging. Essential for building resilient, scalable backend systems with independent services.

CDN (Frontend/Global)

Accelerate content delivery by caching static assets closer to users globally. Critical for performance and user experience in web applications.

Both improve scalability and performance but serve different purposes: MQ for backend communication, CDN for frontend content delivery.



DATA LAKE VS DATA WAREHOUSE

Data Lakes and **Data Warehouses** are both large-scale storage solutions for analytics, but they differ fundamentally in structure, processing approach, and use cases. Lakes store raw data flexibly, while warehouses store processed data in optimized structures.

DATA LAKE (The Reservoir) - Raw & Flexible

A centralized repository that stores massive amounts of raw, unprocessed data in its native format. Accepts any data type without requiring upfront structure. Think of it as a vast reservoir holding everything until needed.

Data Types: Stores ALL data types - structured (databases), semi-structured (JSON, XML, logs), and unstructured (images, videos, text files). No restrictions on format.

Schema-on-Read: Structure is defined when querying data, not when storing. Flexibility to interpret data differently for different use cases.

ELT (Extract, Load, Transform): Load raw data first, transform later when needed. Preserves original data for future unknown use cases.

Cost: Low-cost commodity hardware and object storage (like Amazon S3). Optimized for storing massive volumes cheaply.

Users: Data scientists, data engineers, analysts for exploration, machine learning, and experimentation.

Examples: AWS S3, Azure Data Lake Storage, Google Cloud Storage

DATA WAREHOUSE (The Structured Store) - Curated & Fast

A structured repository optimized for fast queries and reporting. Data is processed, cleaned, and organized into relational tables before storage. Think of it as an organized warehouse with everything labeled and shelved.

Data Types: Stores mainly structured data in relational, modeled format (tables with defined schemas). Data is cleaned and organized.

Schema-on-Write: Structure must be defined before loading data. Ensures data consistency but requires upfront planning and design.

ETL (Extract, Transform, Load): Transform data before loading. Data is cleaned, validated, and structured during the staging process.

Cost: Higher-cost optimized storage with high-performance infrastructure designed for fast analytical queries.

Users: Business analysts and BI professionals for reporting, dashboards, and business intelligence.

Examples: Snowflake, Amazon Redshift, Google BigQuery

Key Differences

Aspect	Data Lake	Data Warehouse
Data Format:	Raw, unprocessed (any format)	Processed, structured (tables)
Schema:	Schema-on-Read (flexible)	Schema-on-Write (rigid)
Processing:	ELT - Load first, transform later	ETL - Transform before loading
Cost:	Lower (commodity storage)	Higher (optimized performance)
Query Speed:	Slower (processing at query time)	Faster (pre-optimized structure)
Use Case:	Exploration, ML, unknown future needs	BI, reporting, known analytical queries

 **Key Takeaway:** Choose based on data type and use case. **Data Lake** for flexibility, raw data exploration, machine learning, and unknown future queries. **Data Warehouse** for structured, high-performance analytics, business intelligence, and reporting. Many modern organizations use both in a complementary "data lakehouse" architecture.

BIG DATA

Big Data refers to datasets so large, complex, and fast-moving that traditional data processing tools cannot handle them effectively. It requires specialized architectures, distributed systems, and powerful tools to capture, store, process, and analyze.

The 3 V's (Core Characteristics)

Volume

(The Amount)

Massive scale of data - terabytes, petabytes, even exabytes. Traditional systems can't store or process this amount. Requires distributed storage across many machines.

Velocity

(The Speed)

Speed at which data is generated, processed, and analyzed. Real-time or near-real-time processing vs. batch. Streaming data from sensors, social media, transactions.

Variety

(The Types)

Different forms of data from various sources - structured (databases), semi-structured (JSON, XML, logs), and unstructured (images, videos, text).

Additional V's: Veracity (data accuracy/trustworthiness), Value (usefulness of insights), Variability (changing data patterns and meanings)

Processing Pipeline: How It Works

1

Data Sources

IoT sensors, social media, transactions, logs - diverse origins

2

Ingestion

Kafka, stream processors collect and route data into the system

3

Storage

Data Lake (raw, flexible) or Data Warehouse (structured, optimized)

4

Processing

Batch (Hadoop, MapReduce) or Stream (Spark, Flink) processing engines

5

Analytics

ML, BI, visualization tools extract insights from processed data

Key Challenges

Storage Cost & Management: Expensive to store petabytes of data. Requires sophisticated management strategies and infrastructure.

Processing Latency: Trade-off between speed and accuracy. Real-time processing vs. batch processing decisions impact architecture.

Data Quality & Governance: Messy, inconsistent data from various sources. Requires cleaning, validation, and governance policies.

Complexity: Distributed systems are inherently complex. Requires specialized expertise in big data technologies and architecture.

Tools & Ecosystem

Storage: Hadoop HDFS, AWS S3, Azure Blob - distributed file systems for massive data volumes

Processing: Apache Spark, Flink, Hive - distributed computing frameworks for batch and stream processing

Messaging: Apache Kafka, RabbitMQ - high-throughput message brokers for data ingestion

NoSQL Databases: Cassandra, HBase, MongoDB - scalable databases for diverse data types

 **Key Takeaway:** Big Data isn't just about size; it's about the architecture to capture, store, process, and analyze massive, diverse, and fast-moving datasets for insights. Requires specialized distributed systems, processing frameworks, and storage solutions. Success depends on choosing the right tools and architectures for your specific Volume, Velocity, and Variety requirements.